

Objetivos da aula:

- Compreender o conceito e a importância dos testes unitários
- Escrever testes básicos utilizando o Jest
- Identificar e criar cenários de teste (casos válidos e de erro)
- Utilizar matchers para validação de resultados
- Aplicar testes em funções síncronas e assíncronas
- Compreender e utilizar stub, mock e spy para isolar dependências

Veja o vídeo para tirar dúvidas: <https://youtu.be/d1o0gE80Bf0>

Repositório: <https://github.com/arleysouza/tpll-2026-1-testes-unitarios>

Índice:

- i. Fundamentos de Testes Unitários
- ii. Testes Unitários com Jest
- iii. Testando com Dependências: Mocks, Stubs e Spies

i. Fundamentos de Testes Unitários

1.1. Introdução

O desenvolvimento de software moderno exige não apenas a implementação de funcionalidades, mas também a garantia de que essas funcionalidades operem corretamente ao longo do tempo. Nesse contexto, os testes desempenham um papel central na qualidade do software.

Entre os diversos níveis de teste existentes, destacam-se os **testes unitários**, que constituem a base de práticas mais avançadas, como testes de integração, testes end-to-end e TDD.

Ao desenvolver um programa, normalmente seguimos este processo:

1. Escrevemos o código
2. Executamos
3. Verificamos se funciona

Agora pense na seguinte situação:

Você implementa uma função hoje, ela funciona perfeitamente.

Uma semana depois, você modifica outra parte do sistema... e sem perceber, quebra algo que já funcionava.

👉 Pergunta:

Como garantir que o sistema continua funcionando após mudanças?

Resposta: testes

Testes são mecanismos que permitem verificar automaticamente se o sistema está funcionando como esperado.

Em vez de testar manualmente toda vez:

O próprio código valida o comportamento do sistema.

1.2. Definição de Teste Unitário

Um teste unitário consiste na verificação da menor unidade testável de um sistema.

Mas o que é “uma unidade”?

Em programação, normalmente é:

- Uma função
- Um método
- Um pequeno módulo

Exemplo simples

```
function somar(a: number, b: number): number {  
  return a + b;  
}
```

Essa função é uma **unidade**.

O que queremos testar?

Pergunta: Se eu passar 2 e 3, o resultado é 5?

Se sim → está correto

Se não → há erro

1.3. Como pensar um teste

Antes de escrever código, pense assim:

“Dada uma entrada, qual deve ser a saída?”

Exemplo de entrada:

a = 2, b = 3

Saída esperada: 5

Estrutura mental de um teste

Todo teste segue essa lógica:

1. Preparar os dados
2. Executar a função
3. Verificar o resultado

1.4. Padrão AAA (Arrange, Act, Assert)

Arrange (Preparar) Definir os dados de entrada

```
const a = 2;  
const b = 3;
```

Act (Executar) Chamar a função

```
const resultado = somar(a, b);
```

Assert (Verificar) Comparar o resultado com o esperado

```
expect(resultado).toBe(5);
```

1.5. O que torna um bom teste?

Agora que você entendeu o básico, vamos melhorar a qualidade.

1.5.1 Isolamento

Um teste deve avaliar apenas uma coisa.

👉 Exemplo ruim:

- Função depende de banco
- Depende de API
- Depende de outro módulo

👉 Isso dificulta saber onde está o erro.

1.5.2 Determinismo

O teste deve sempre dar o mesmo resultado.

Evite:

- Datas atuais (`new Date()`)
- Números aleatórios
- Dados externos

1.5.3 Simplicidade

Um bom teste deve ser:

- Fácil de ler
- Fácil de entender
- Curto

Se o teste é complexo, algo está errado.

1.5.4 Independência

Um teste não deve depender de outro.

👉 Você deve conseguir rodar qualquer teste sozinho.

1.5.5 Clareza

O nome do teste deve explicar o que está sendo verificado.

Exemplo:

```
"deve somar dois números positivos"
```

1.6. O que testar

Um bom conjunto de testes deve cobrir:

- **Fluxos principais:**
Casos mais comuns de uso.
- **Casos de borda:**
Situações limite:
 - Valores nulos
 - Valores extremos
 - Entradas inesperadas
- **Tratamento de erros**
Verificação de exceções e comportamentos inválidos.

1.7. Benefícios dos testes unitários

A adoção de testes unitários traz diversas vantagens:

- Redução de bugs em produção
- Maior confiabilidade do sistema
- Facilidade de manutenção
- Segurança para refatorações
- Documentação viva do comportamento

1.8. Relação com qualidade de software

Um código difícil de testar frequentemente apresenta problemas estruturais, tais como:

- Alto acoplamento
- Baixa coesão
- Dependências excessivas

Portanto, os testes unitários também funcionam como um **indicador de qualidade arquitetural**.

ii. Testes Unitários com Jest

2.1. Introdução

O Jest é um framework de testes amplamente utilizado no ecossistema JavaScript/TypeScript. Ele fornece:

- Execução de testes
- Funções de asserção
- Ferramentas para simulação (mocks)
- Relatórios detalhados

2.2. Estrutura básica de um teste

Essa função recebe dois números e retorna a soma.

```
function somar(a: number, b: number): number {  
  return a + b;  
}
```

No Jest, um teste é definido pela função `test` ou `it` (só muda o nome):

```
test("descrição do teste", () => {  
  expect(valorObtido).toBe(valorEsperado);  
});
```

Exemplo:

```
test("deve somar dois números positivos", () => {  
  expect(somar(2, 3)).toBe(5);  
});
```

Agora vamos entender linha por linha.

- `test(...)` ou `it(...)`

```
test("deve somar dois números positivos", () => { ... });
```

- Define um teste
- O primeiro parâmetro é uma **descrição**
- O segundo é uma função que contém o teste

👉 Pense como:

“Vou verificar se algo funciona corretamente”

- `expect(...)`

```
expect(somar(2, 3))
```

- Executa a função `somar`
- Captura o resultado

👉 Aqui estamos dizendo:

“Quero verificar o resultado de `somar(2, 3)`”

- `toBe(...)`

```
.toBe(5)
```

- Define o valor esperado
- Compara com o resultado

👉 Tradução completa do teste:

“Espero que `somar(2, 3)` seja igual a `5`”

2.3. Pensando como um testador

“Como saber o que testar?”

Resposta:

Pense em cenários

Quais cenários são relevantes no teste da função `somar`?

1. Valores positivos
2. Valores negativos
3. Zero

Testes completos

```
test("deve somar valores positivos", () => {  
  expect(somar(2, 3)).toBe(5);  
});
```

```
test("deve somar valores negativos", () => {  
  expect(somar(-2, -3)).toBe(-5);  
});
```

```
test("deve somar valores zeros", () => {  
  expect(somar(0, 0)).toBe(0);  
});
```

👉 Cada teste verifica **um comportamento específico**

2.4. Regra fundamental

Um teste deve validar apenas um comportamento

Deve-se evitar múltiplas verificações no mesmo teste:

```
test("teste de soma", () => {  
  expect(somar(2, 3)).toBe(5);  
  expect(somar(-2, -3)).toBe(-5);  
  expect(somar(0, 0)).toBe(0);  
});
```

Problema:

Se falhar, não sabemos qual caso quebrou

2.5. test vs it

```
it("deve retornar 5 ao somar 2 e 3", () => {
  expect(somar(2, 3)).toBe(5);
});
```

👉 Leitura natural:

“Ele deve retornar 5 ao somar 2 e 3”

"It should return 5 when adding 2 and 3"

O `it` não significa literalmente “teste individual” (apesar de algumas pessoas entenderem assim), mas sim a forma reduzida de “it should...”, tornando a especificação dos testes mais legível e natural.

2.6. Matchers - como comparar valores

Matchers são as formas de verificar resultados.

- Comparação de valores simples

```
expect(5).toBe(5);
```

👉 Igualdade exata usando `===`

- Comparação de objetos

```
expect({ nome: "Ana" }).toEqual({ nome: "Ana" });
```

👉 `toBe` não funciona bem com objetos

👉 `toEqual` compara conteúdo

- Números decimais

Problema com número decimais

```
expect(0.1 + 0.2).toBe(0.3); // pode falhar
```

👉 JavaScript tem imprecisão com floats

Solução

```
expect(0.1 + 0.2).toBeCloseTo(0.3);
```

- Comparações numéricas

```
expect(5).toBeLessThan(10);
```

```
expect(10).toBeGreaterThanOrEqual(10);
```

- Valores especiais

```
expect(null).toBeNull();
```

```
expect(undefined).toBeUndefined();
```

```
expect(true).toBeTruthy();
```

```
expect(false).toBeFalsy();
```

- Strings

```
expect("Teste com Jest").toMatch(/Jest/);
```

- Exceções

```
function dividir(a: number, b: number) {
  if (b === 0) throw new Error("Divisão por zero");
  return a / b;
}

test("deve lançar erro ao dividir por zero", () => {
  expect(() => dividir(10, 0)).toThrow("Divisão por zero");
});
```

👉 Importante: A função precisa estar dentro de outra função:

```
() => dividir(10, 0)
```

Caso contrário, o erro ocorre antes do teste capturar.

- Testes assíncronos

Algumas funções não retornam valores diretamente.

Exemplo

```
async function dobro(n: number): Promise<number> {
  if (n < 0) throw new Error("Número inválido");
  return n * 2;
}
```

Teste

```
test("deve retornar o dobro", async () => {
  await expect(dobro(2)).resolves.toBe(4);
});

test("deve lançar exceção", async () => {
  await expect(dobro(-1)).rejects.toThrow("Número inválido");
});
```

Entendimento

- `async` → função retorna Promise
- `await` → espera o resultado
- `resolves` → verifica sucesso
- `rejects` → verifica falha
- Modificador de negação
`expect(5).not.toBe("5");`

2.7. Organizando testes com describe

```
describe("operações matemáticas", () => {
  test("deve fazer a operação de soma", () => {
    expect(2 + 3).toBe(5);
  });
});
```

Vantagens

- Organiza os testes
- Facilita a leitura
- Agrupa por contexto


```
test("deve fazer a operação de subtração", () => {  
  expect(5 - 3).toBe(2);  
});  
});
```

2.9. Hooks (preparação de testes)

Às vezes precisamos executar algo antes dos testes.

```
beforeEach(() => {  
  console.log("Executado antes de cada teste");  
});
```

Uso típico

- Resetar variáveis
- Preparar dados
- Limpar estado

Principais hooks

- beforeAll
- afterAll
- beforeEach
- afterEach

iii. Testando com Dependências: Mocks, Stubs e Spies

3.1. Introdução

Até agora, testamos funções como:

```
function somar(a: number, b: number): number {  
  return a + b;  
}
```

Essas funções são fáceis de testar porque:

- Não dependem de nada externo
- Sempre retornam o mesmo resultado
- São totalmente previsíveis

O que muda em sistemas reais?

Agora considere:

```
function buscarUsuario(id: number) {  
  return { id, nome: "João" };  
}  
  
function obterNomeUsuario(id: number): string {
```

```
    const usuario = buscarUsuario(id);  
    return usuario.nome;  
}
```

Pergunta importante

Queremos testar:

```
obterNomeUsuario(1)
```

Mas surge um problema:

Estamos testando obterNomeUsuario ou buscarUsuario?

3.2. Conceito de dependência

A função buscarUsuario é uma:

dependência

Ou seja:

Algo que a função precisa para funcionar, mas não controla.

Por que isso é um problema?

Se a dependência:

- Falhar
- Mudar comportamento
- Acessar banco de dados
- Chamar uma API

👉 O teste deixa de ser confiável.

Regra fundamental

Um teste unitário deve testar apenas uma unidade

Como resolver isso?

A ideia é simples:

Substituir a dependência real por uma versão controlada

Tornando a função testável

```
function obterNomeUsuario(buscar: Function, id: number): string {  
    const usuario = buscar(id);  
    return usuario.nome;  
}
```

Agora podemos controlar a dependência.

Test Doubles

Essas substituições são chamadas de:

Test Doubles

Os três principais tipos são:

- Stub
- Mock
- Spy

3.3. Stub — Controlando o retorno

Pergunta que estamos respondendo

“E se eu quiser decidir exatamente o que a dependência retorna?”

Ideia

Não queremos executar a função real.

Queremos **forçar um retorno específico**.

Exemplo

```
test("deve retornar o nome corretamente", () => {  
  const buscarStub = jest.fn().mockReturnValue({ nome: "Maria" });  
  
  const resultado = obterNomeUsuario(buscarStub, 1);  
  
  expect(resultado).toBe("Maria");  
});
```

Explicação passo a passo

- Criamos uma função falsa (jest.fn())
- Definimos o retorno (mockReturnValue)
- Substituímos a função real
- Testamos apenas a lógica

Resumo

Stub = “Eu controlo o que a dependência retorna”

3.4. Mock — Verificando comportamento

Pergunta que estamos respondendo

“A função chamou a dependência corretamente?”

Exemplo

```
function processar(callback: Function) {  
  callback("dados");  
}
```

```
}
```

Teste

```
test("deve chamar callback corretamente", () => {  
  const mock = jest.fn();  
  
  processar(mock);  
  
  expect(mock).toHaveBeenCalledWith("dados");  
});
```

Explicação

- Criamos uma função falsa
- Executamos o código
- Verificamos como a função foi usada

Resumo

Mock = “Quero verificar como a função foi utilizada”

3.4. Spy — Observando sem substituir

Pergunta que estamos respondendo

“Quero observar uma função real sem alterá-la”

Exemplo

```
const calculadora = {  
  somar(a: number, b: number) {  
    return a + b;  
  },  
};
```

Teste

```
test("deve observar chamada", () => {  
  const spy = jest.spyOn(calculadora, "somar");  
  
  calculadora.somar(2, 3);  
  
  expect(spy).toHaveBeenCalledWith(2, 3);  
});
```

Explicação

- A função continua funcionando normalmente
- Apenas registramos as chamadas

Resumo

Spy = “Vou observar o comportamento sem interferir”

3.5 Comparação final

Tipo	Pergunta que responde
Stub	O que a função deve retornar?
Mock	A função foi chamada corretamente?
Spy	Como a função está sendo usada?

Exercícios

Exercício 1 – Testes com Jest

Função

```
export function calcularMedia(notas: number[]): number {  
  if (notas.length === 0) {  
    throw new Error("Lista de notas vazia");  
  }  
  
  const soma = notas.reduce((acc, n) => acc + n, 0);  
  return soma / notas.length;  
}
```

Tarefas

Escreva testes unitários para os seguintes cenários:

1. Média de valores inteiros
 - Ex.: [10, 8, 6] → 8
2. Média com valores decimais
 - Ex.: [7.5, 8.5] → 8
3. Lista com um único valor
 - Ex.: [10] → 10
4. Lista com zeros
 - Ex.: [0, 0, 0] → 0
5. Lista vazia
 - Deve lançar erro

Requisitos

- Um teste por cenário
- Usar describe
- Utilizar matchers adequados (toBe, toBeCloseTo, toThrow)
- A função calcularMedia deve estar no arquivo /src/exercicio1.ts
- Os testes deverão estar no arquivo /test/exercicio1.test.ts

Exercício 2 – Testes com Objetos e Validações**Função**

```
type Usuario = {  
  nome: string;  
  idade: number;  
};  
  
export function criarUsuario(nome: string, idade: number): Usuario {  
  if (!nome) {  
    throw new Error("Nome obrigatório");  
  }  
  
  if (idade < 0) {  
    throw new Error("Idade inválida");  
  }  
  
  return { nome, idade };  
}
```

Tarefas

Escreva testes unitários para os seguintes cenários:

1. Criação válida de usuário
 - Ex.: "Ana", 20 → { nome: "Ana", idade: 20 }
2. Nome vazio
 - Deve lançar erro
3. Idade negativa
 - Deve lançar erro
4. Nome com espaços (ex.: "João")
 - Deve funcionar corretamente

Requisitos

- Utilizar describe
- Um teste por cenário
- Usar:
 - toEqual para objetos
 - toThrow para erros
- O código a ser testado deve estar no arquivo /src/exercicio2.ts
- Os testes deverão estar no arquivo /test/exercicio2.test.ts

Exercício 3 — Testes com Funções Assíncronas

Função

```
export async function dobro(n: number): Promise<number> {  
  if (n < 0) {  
    throw new Error("Número inválido");  
  }  
  
  return n * 2;  
}
```

Tarefas

Escreva testes unitários para os seguintes cenários:

1. Número positivo
 - Ex: 2 → 4
2. Zero
 - Ex: 0 → 0
3. Número negativo
 - Deve lançar erro

Requisitos

- Utilizar describe
- Um teste por cenário
- Usar:
 - resolves.toBe para casos válidos
 - rejects.toThrow para erro
- O código a ser testado deve estar no arquivo /src/exercicio3.ts
- Os testes deverão estar no arquivo /test/exercicio3.test.ts

Exercício 4 — Testando com Stub

Função

```
export function calcularFrete(  
  obterTaxaFrete: (cep: string) => number,  
  cep: string,  
  valorPedido: number,  
): number {  
  const taxa = obterTaxaFrete(cep);  
  return valorPedido + taxa;  
}
```

Tarefas

Escreva um teste unitário para verificar o seguinte cenário:

1. O valor final do pedido deve ser calculado corretamente a partir do valor do pedido e da taxa de frete retornada pela dependência.

Exemplo esperado:

```
valorPedido = 100
taxaFrete = 20
resultado = 120
```

Requisitos

- Utilizar `jest.fn()`
- Definir o retorno da dependência com `mockReturnValue`
- Verificar o resultado com `toBe`
- O código a ser testado deve estar no arquivo `/src/exercicio4.ts`
- Os testes deverão estar no arquivo `/test/exercicio4.test.ts`

Exercício 5 — Testando com Mock

Função

```
export function registrarLog(
  salvarLog: (mensagem: string) => void,
  mensagem: string,
): void {
  salvarLog(mensagem);
}
```

Tarefas

Escreva um teste unitário para verificar o seguinte cenário:

1. A função deve chamar a dependência `salvarLog` com a mensagem informada.

Exemplo esperado:

```
mensagem = "Usuário autenticado"
```

A dependência deve ser chamada com esse mesmo valor.

Requisitos

- Utilizar `jest.fn()`
- Chamar a função `registrarLog`
- Verificar a chamada com `toHaveBeenCalled`
- O código a ser testado deve estar no arquivo `/src/exercicio5.ts`
- Os testes deverão estar no arquivo `/test/exercicio5.test.ts`

Exercício 6 — Testando com Spy

Função

```
export const calculadora = {
  somar(a: number, b: number): number {
    return a + b;
  },
}
```



```
};  
  
export function executarSoma(a: number, b: number): number {  
  return calculadora.somar(a, b);  
}
```

Tarefas

Escreva um teste unitário para verificar o seguinte cenário:

1. A função `executarSoma` deve chamar o método `calculadora.somar` com os parâmetros corretos.

Exemplo esperado:

```
executarSoma(2, 3)
```

A função `somar` deve ser chamada com 2 e 3.

Requisitos

- Utilizar `jest.spyOn`
- Executar a função `executarSoma`
- Verificar a chamada com `toHaveBeenCalled`
- O código a ser testado deve estar no arquivo `/src/exercicio6.ts`
- Os testes deverão estar no arquivo `/test/exercicio6.test.ts`